

```
!pip install ultralytics opencv-python-headless
```

Collecting ultralytics

Downloading ultralytics-8.4.154-py3-none-any.whl.metadata (46 kB)  
46.5/46.5 kB 564.7 kB/s eta 0:00:00

Requirement already satisfied: opencv-python-headless in /usr/local/lib/python3.13/dist-packages (5.0.0.93)  
Requirement already satisfied: cloudpickle>=3.1.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.1.2)  
Requirement already satisfied: filelock>=3.16.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.32.5)  
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.1.3)  
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.10.0)  
Requirement already satisfied: opencv-python!=4.13.0.90,>=4.7.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (5.0.0.93)  
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (11.3.0)  
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (6.0.3)  
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.32.4)  
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.11.0+cpu)  
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (0.26.0+cpu)  
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (5.9.5)  
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (1.35.2)

Collecting nvidia-ml-py>=12.0.0 (from ultralytics)

Downloading nvidia\_ml\_py-13.610.43-py3-none-any.whl.metadata (9.7 kB)

Collecting ultralytics-thop>=2.1.6 (from ultralytics)

Downloading ultralytics\_thop-2.1.6-py3-none-any.whl.metadata (13 kB)

Collecting ultralytics-platform>=0.1.45 (from ultralytics)

Downloading ultralytics\_platform-0.1.46-py3-none-any.whl.metadata (11 kB)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.3.3)  
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)  
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (4.64.0)  
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.5.1)  
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (26.3)  
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (3.3.2)  
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from matplotlib>=3.3.0->ultralytics) (2.9.0)  
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.13/dist-packages (from polars>=0.20.0->ultralytics) (1.35.2)  
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (3.4.0)  
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (3.19)  
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (2.5.0)  
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests>=2.23.0->ultralytics) (2026.7.22)  
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (4.16.1)  
Requirement already satisfied: setuptools<82 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (80.10.2)  
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (1.14.0)  
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (3.6.1)  
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (3.1.6)  
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.13/dist-packages (from torch>=1.8.0->ultralytics) (2025.12.0)  
Requirement already satisfied: httpx<1,>=0.28 in /usr/local/lib/python3.13/dist-packages (from ultralytics-platform>=0.1.45->ultralytics) (0.28.1)  
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.28->ultralytics-platform>=0.1.45->ultralytics) (4.12.0)  
Requirement already satisfied: httpcore==1.\* in /usr/local/lib/python3.13/dist-packages (from httpx<1,>=0.28->ultralytics-platform>=0.1.45->ultralytics) (1.0.7)  
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.\*->httpx<1,>=0.28->ultralytics-platform>=0.1.45->ultralytics) (0.14.0)  
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.7->matplotlib>=3.3.0->ultralytics) (1.17.0)  
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from sympy>=1.13.3->torch>=1.8.0->ultralytics) (1.3.0)  
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from Jinja2->torch>=1.8.0->ultralytics) (3.0.3)  
Downloading ultralytics-8.4.154-py3-none-any.whl (1.4 MB)  
1.4/1.4 MB 6.7 MB/s eta 0:00:00

Downloading nvidia\_ml\_py-13.610.43-py3-none-any.whl (53 kB)

53.2/53.2 kB 3.8 MB/s eta 0:00:00

Downloading ultralytics\_platform-0.1.46-py3-none-any.whl (78 kB)

78.1/78.1 kB 5.0 MB/s eta 0:00:00

Downloading ultralytics\_thop-2.1.6-py3-none-any.whl (30 kB)

Installing collected packages: nvidia-ml-py, ultralytics-thop, ultralytics-platform, ultralytics

Successfully installed nvidia-ml-py-13.610.43 ultralytics-8.4.154 ultralytics-platform-0.1.46 ultralytics-thop-2.1.6

```
from ultralytics import YOLO
import cv2
import matplotlib.pyplot as plt
```

Creating new Ultralytics Settings v0.0.8 file ☒

View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'

Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs\_dir=path/to/dir'. For help see <https://docs.ultralytics.com/usage>

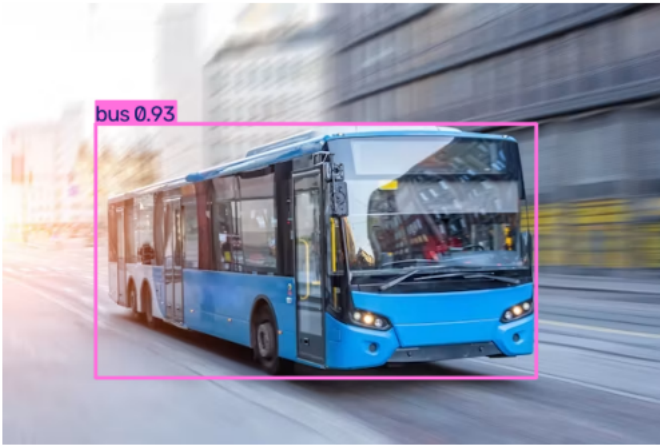
```
model = YOLO("yolov8n.pt")
```

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt> to 'yolov8n.pt': 100% — 6.2MB 22.8MB/s 0.

```
url1 = "/content/premium_photo-1664302152991-d013ff125f3f.avif"
url2 = "/content/images (2).jpg"
```

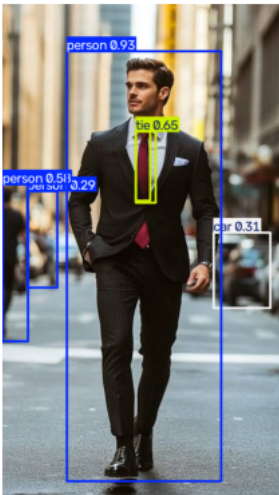
```
results1 = model(url1)
img1 = cv2.cvtColor(results1[0].plot(), cv2.COLOR_BGR2RGB)
plt.imshow(img1)
plt.axis('off')
plt.show()
```

image 1/1 /content/premium\_photo-1664302152991-d013ff125f3f.avif: 448x640 1 bus, 361.7ms  
Speed: 31.0ms preprocess, 361.7ms inference, 43.3ms postprocess per image at shape (1, 3, 448, 640)



```
results2 = model(url2)
img2 = cv2.cvtColor(results2[0].plot(), cv2.COLOR_BGR2RGB)
plt.imshow(img2)
plt.axis('off')
plt.show()
```

image 1/1 /content/images (2).jpg: 640x384 3 persons, 1 car, 2 ties, 119.3ms  
Speed: 5.9ms preprocess, 119.3ms inference, 1.1ms postprocess per image at shape (1, 3, 640, 384)



```
for box in results1[0].boxes:
    print(model.names[int(box.cls[0])], float(box.conf[0]))
```

bus 0.9304298758506775

```
for box in results2[0].boxes:
    print(model.names[int(box.cls[0])], float(box.conf[0]))
```

person 0.925821840763092  
tie 0.6459293365478516  
person 0.581556499004364  
tie 0.5207036137580872  
car 0.31368085741996765  
person 0.28730693459510803

## PART - B

1. Differentiate between image classification, object detection, and image segmentation. Image classification assigns a single label to an entire image. Object detection identifies multiple objects and localizes each with a bounding box along with a class label. Image segmentation goes further by classifying every pixel in the image, producing a pixel-wise mask for each object or region.
2. Explain the working principle of the YOLO (You Only Look Once) object detection algorithm. YOLO divides an image into a grid and predicts bounding boxes, objectness scores, and class probabilities for each grid cell in a single forward pass through the network. This unified approach allows it to detect and classify objects simultaneously, making it extremely fast compared to methods that use separate region proposal and classification stages.
3. Compare YOLO, SSD, and Faster R-CNN based on speed, accuracy, and practical applications. YOLO offers the fastest inference speed and is ideal for real-time applications like autonomous driving and video surveillance, though it can be slightly less accurate

on small objects. SSD balances speed and accuracy, using multiple feature maps at different scales, making it suitable for mobile and embedded systems. Faster R-CNN achieves the highest accuracy through a two-stage region proposal and classification process but is slower, making it more suitable for applications where accuracy matters more than speed, such as medical imaging.

4. What is a bounding box? Why is it important in object detection? A bounding box is a rectangular region defined by coordinates (typically x, y, width, height or corner points) that tightly encloses an object within an image. It is important because it provides the spatial location of an object, enabling systems to know not just what an object is but also where it is positioned in the image.
5. Explain the significance of confidence score and Intersection over Union (IoU) in object detection. The confidence score indicates how certain the model is that a detected bounding box contains an object of a particular class, helping filter out low-quality or false detections. IoU measures the overlap between a predicted bounding box and the ground truth box by dividing their intersection area by their union area; it is used to evaluate detection accuracy and determine whether a prediction counts as a correct detection.
6. What are the advantages of using pre-trained deep learning models over traditional image processing techniques? Pre-trained models are trained on massive datasets and can generalize well to new images without requiring extensive labeled data or manual feature engineering. They save significant training time and computational resources, achieve higher accuracy, and can be fine-tuned for specific tasks through transfer learning, unlike traditional techniques that rely on hand-crafted features and rules.
7. Why are datasets such as MS COCO and Pascal VOC widely used for object detection research? These datasets contain large numbers of diverse, accurately annotated images covering many object categories in varied real-world scenarios, making them reliable benchmarks for training and evaluating models. Their standardized annotations and widespread adoption allow researchers to fairly compare the performance of different object detection algorithms.
8. Mention five real-world applications of object detection in computer vision. Autonomous vehicles for detecting pedestrians, vehicles, and traffic signs; surveillance systems for monitoring and security; healthcare for identifying tumors or abnormalities in medical scans; retail analytics for customer behavior tracking and inventory management; and industrial inspection for detecting defects on production lines.
9. What challenges are commonly encountered while detecting objects in complex real-world environments? Challenges include occlusion where objects are partially hidden behind others, variations in lighting and weather conditions, scale variation where objects appear at different sizes, cluttered backgrounds that make object boundaries unclear, and detecting small or overlapping objects accurately in crowded scenes.
10. How can object detection systems be further improved using recent advancements in deep learning and computer vision? Improvements can come from using transformer-based architectures like DETR for better global context understanding, employing advanced data augmentation and synthetic data generation to handle diverse scenarios, using lightweight and efficient architectures for edge deployment, incorporating self-supervised and semi-supervised learning to reduce dependency on labeled data, and applying multi-modal sensor fusion combining vision with LiDAR or radar data for more robust detection.

...

Ellipsis